EXPERIMENT 12 : MINIMAX ALGORITHM FOR GAMING

AIM

To develop a Python program that implements the Minimax Algorithm for game playing and determines the optimal move for a player in a two-player zero-sum game.

OBJECTIVE

- To understand the concept of the Minimax Algorithm in Artificial Intelligence.
- To simulate decision-making in two-player games.
- To maximize the player's score while minimizing the opponent's score.
- To determine the optimal move from the game tree.

ALGORITHM

Step 1: Start.

Step 2: Construct the game tree.

Step 3: Assign utility values to the terminal nodes.

Step 4: Start from the leaf nodes.

Step 5: At the **MAX** level, choose the maximum value.

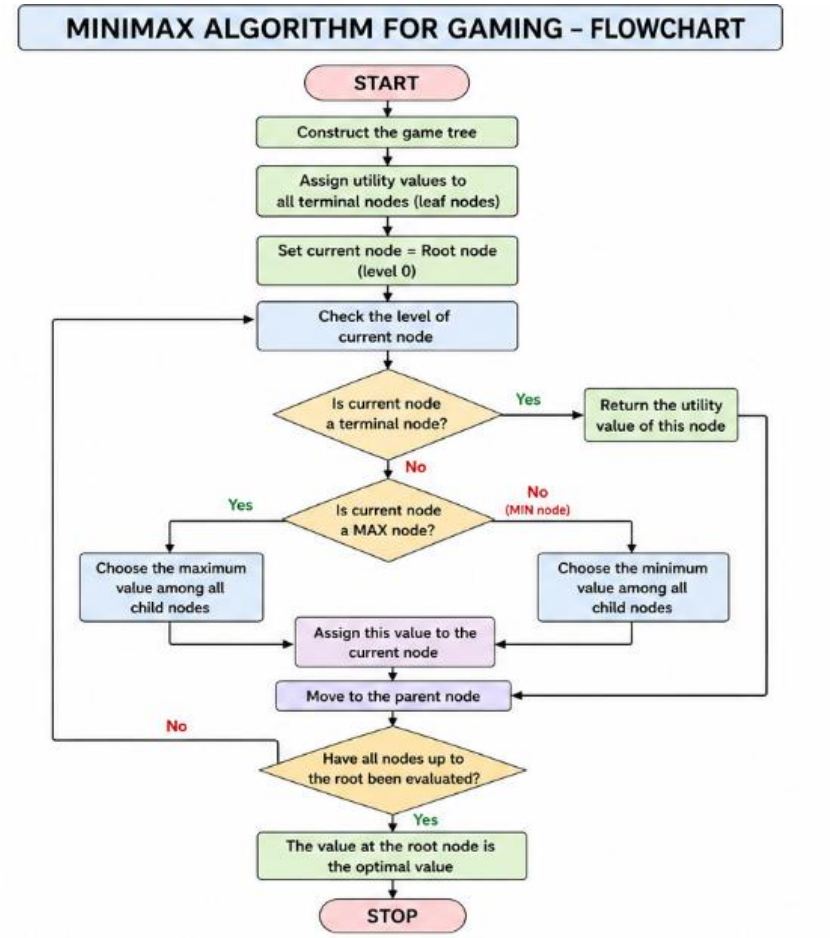
Step 6: At the **MIN** level, choose the minimum value.

Step 7: Continue until the root node is evaluated.

Step 8: Display the optimal move.

Step 9: Stop.

FLOW CHART



SOURCE CODE

```
import math


def minimax(depth, nodeIndex, maximizingPlayer, values, height):
    if depth == height:
        return values[nodeIndex]


    if maximizingPlayer:
        return max(
            minimax(depth + 1, nodeIndex * 2, False, values, height),
```

```
            minimax(depth + 1, nodeIndex * 2 + 1, False, values, height)
        )
    else:
        return min(
            minimax(depth + 1, nodeIndex * 2, True, values, height),
            minimax(depth + 1, nodeIndex * 2 + 1, True, values, height)
        )

values = [3, 5, 2, 9, 12, 5, 23, 23]
height = 3
print("Optimal Value:", minimax(0, 0, True, values, height))
```

OUTPUT

```
Optimal Value: 12
```

RESULT

The Python program successfully implements the Minimax Algorithm and determines the optimal move by maximizing the player's score while minimizing the opponent's score.